

MEMO

To: Manager

From: Yassine

Re: Investigation: High Mileage EV Overpricing — Analysis made with Python

1. Problem Definition

The complaint raised concerns that high mileage EVs are being predicted too high. Overpricing is defined as cases where the guarantee price exceeds the realized sold price, measured as spread in DKK. A positive spread represents direct financial leakage, and a negative spread represents under-commitment, which carries commercial and trust risks. The investigation follows three steps: confirm the issue with data, identify the root cause, and recommend corrective actions and ongoing monitoring.

```
df['spread'] = df['guarantee_price_dkk'] - df['sold_price_dkk']
```

2. Data Analysis — Key Findings

EVs confirmed as the problem segment: The analysis confirms that EVs represent a high average spread of 6,655.17 DKK compared to Petrol / Diesel / Hybrid ranging between -300 and 400 DKK (Figure 1).

```
df.groupby('fuel_type')['spread'].agg(['mean', 'sum', 'count'])
```

High mileage EVs: The analysis confirms that high mileage EVs are predicted high, representing 13,857 DKK average spread compared to low mileage EVs with -67 DKK (Figure 2).

```
threshold = ev_df['mileage_km'].median()  
ev_df['mileage_band'] = np.where(  
    ev_df['mileage_km'] <= threshold, 'Low Mileage', 'High Mileage'  
)
```

Method and reasoning: Spread was calculated per transaction as guarantee price minus sold price, then aggregated by fuel type to test whether the issue is isolated to EVs or appears across other fuel types. To confirm whether high mileage EVs were the only ones affected, I split the EV population at the median mileage of 83,000 km, chosen because it comes from the data and gives two balanced groups.

Narrowing the source of the overpricing:

- **Robot vs analyst:** On high mileage EVs, robot-sourced prices show an average spread of 30,167 DKK versus 1,625 DKK for analyst-sourced prices. The robot accounts for 181,000 DKK of the 194,000 DKK total leakage despite pricing fewer cars (Figure 3).
- **Make and model:** All three Tesla models dominate: Model S at 36,500 DKK, Model Y at 29,000 DKK, Model 3 at 18,667 DKK. Polestar 2 and Volkswagen ID.4 sit at 2,500 DKK, while Hyundai, Kia and Nissan show negative spread (Figure 4).
- **Model version:** Both versions show almost identical spreads of 14,125 DKK and 13,500 DKK, indicating a structural issue in the pricing logic rather than one introduced by a version update (Figure 5).

```
ev_df[ev_df['mileage_band'] == 'High Mileage'] \  
    .groupby('guarantee_price_source')['spread'] \  
    .agg(['mean', 'sum', 'count'])
```

Method and reasoning: I compared robot and analyst prices on the same set of high mileage EVs because both operate under identical market conditions, so any difference points to the pricing method, not the market. I then segmented by make and model to test whether the overpricing was concentrated or spread evenly, and by model version to check whether a recent update had introduced it. Together these steps narrowed the issue from “EVs are overpriced” to “the robot systematically overprices high mileage Teslas across all versions”, which directed the investigation to the pricing logic itself.

3. Root Cause — Python Script Inspection

The analysis points to a segment-specific, model-related issue rather than a data quality or integration problem, so the next step was to inspect the pricing script.

The main issue: additions do not scale, the correction does

- Every EV gets 24,000 DKK added, plus 14,000 DKK if it is a Tesla or Polestar. These are the same for every car regardless of mileage.
- The only correction is 0.18 DKK per km above 80,000 km.
- On a 50,000 km EV: 38,000 DKK added, nothing taken off.
- On a 130,000 km EV: the same 38,000 DKK added, only 9,000 DKK taken off.
- The higher the mileage, the further the price drifts from what the car is actually worth. This matches the pattern in the data, where low mileage EVs average -67 DKK spread and high mileage EVs 13,857 DKK.

```
price += 24_000
price -= max(features["mileage_km"] - 80_000, 0) * 0.18

if features["make"] in {"tesla", "polestar"}:
    price += 14_000
```

A second issue: the power penalty uses the wrong unit

- The payload sends power_kw (kilowatts), but the feature is named horse_power and compared against a threshold of 260.
- In the sample payload, 239 kW passes without penalty, though 239 kW is roughly 320 HP.
- This makes the penalty under-apply. It affects EVs generally rather than high mileage specifically, so it is not the cause of this issue, but it should be corrected.

```
"horse_power": float(payload.get("power_kw") or 0)
...
price -= max(features["horse_power"] - 260, 0) * 120
```

Immediate priorities

- Recalibrate the high mileage EV penalty
- Make the brand premium conditional on mileage band

Verdict: the overpricing is confirmed, and the cause sits in the pricing logic, not the data or the market — brand and baseline premiums do not reduce with mileage, and the mileage correction is too weak to offset them.

4. Monitoring Suggestion

My recommendation is to build a weekly monitoring function that tracks the spread (guarantee price minus sold price) for each segment, comparing it against that segment's own recent history. Segments are

defined by market, fuel type, mileage band and make. Each gets its own control limits from roughly six months of its own past weeks, with an alert when the current week moves more than two standard deviations from that mean. The limits are set per segment because segments behave differently, so one threshold across all segments would not work.

```
band['upper_limit'] = band['mean'] + n_std * band['std']  
band['lower_limit'] = band['mean'] - n_std * band['std']
```

One design decision matters more than the rest. A plain rolling baseline eventually absorbs a lasting problem into its own reference. The limits recalculate around the broken level, the current week looks unremarkable, and the alerts go quiet while money keeps leaking. Any week already flagged is therefore excluded when the baseline is next built.

```
baseline = weekly[  
    weekly['year_week'].isin(window_weeks) &  
    (weekly['flagged'] == False)  
]
```

The check returns two lists: segments that breached, and segments with too little clean history to judge, which surfaces new markets and makes before they go unmonitored. Healthy segments stay off the list so real signals are not buried.

One caveat: the reference period must be verified as genuinely healthy before going live. In this dataset the Tesla overpricing is present in the earliest week, so limits built from it would treat the problem as normal and never fire an alert. See the appendix for this and other limitations.

Detection logic is defined in the attached notebook. Scheduling, database connection and alert delivery would be built with the data engineering team.